

Stage 2 Report

CS 4366: Senior Capstone Project

The Clever Coders

Robert Wells – Team Leader

Sang Nguyen – Documentation Specialist

Rocco Swaney – Backend Developer

Gage Flores – Frontend Developer

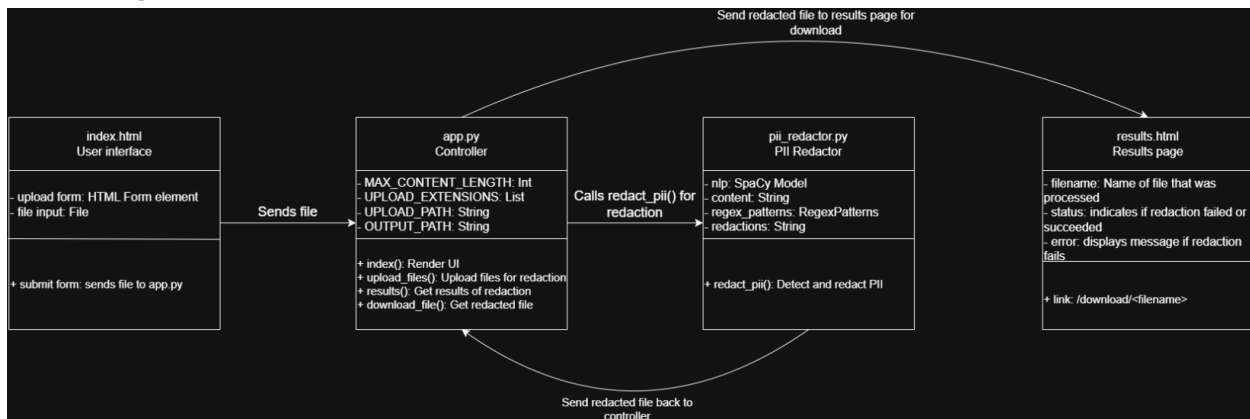
Logan LiVigni – Website Developer

Project Description:

The project we decided to tackle this semester is personally identifiable information (PII) redaction from text documents. IBM says, “Personally identifiable information (PII) is any information connected to a specific individual that can be used to uncover or steal that individual’s identity.” These include information such as social security numbers, full name, email address, or phone number. The security benefits of redacting PII include preventing identity theft and fraud, ensuring regulatory compliance, mitigates insider threats, protects against data leaks, enhances document security in cloud storage, and boosts consumer and client trust. Our objective during this project is to develop a program that will automatically detect and redact PII to protect an individual’s privacy.

UML Diagrams:

Class Diagram:



This is our class diagram for our four-component flow in our PII redaction system. It involves uploading a file, redacting PII, and returning redacted file for download.

1. Index.html

- This is the user interface
- It has an upload form where users can upload file(s) for redaction
- When the user submits, it sends the file to the controller, app.py

2. App.py

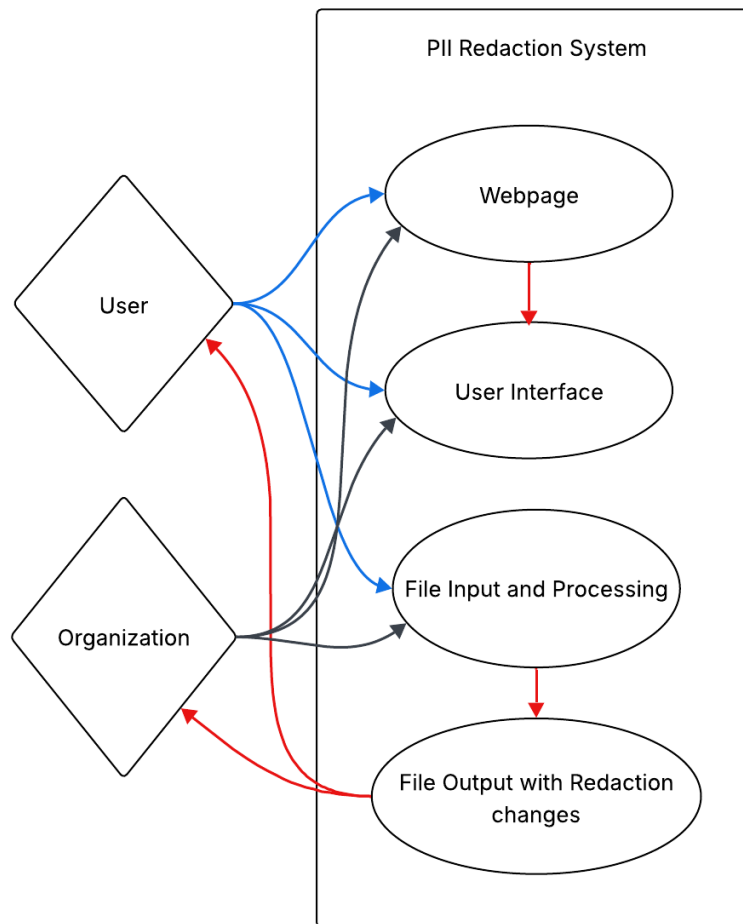
- This is the controller of our system which controls system flow
- It receives files, makes sure they are valid length and extension, and saves them to upload folder
- Then it calls redaction function, which is in pii_redactor.py
- After redaction, it renders results.html where we can see any errors and download the redacted file

3. Pii_redactor.py

- This is the functionality of our system
- Handles text-processing and PII detection and redaction

- Uses SpaCy and regex patterns for PII redaction
 - Returns output file path and control back to app.py
4. Results.html
- This is the result page for our system
 - It displays the outcome of the redaction
 - Shows the redacted filename and a link for downloading it

Use Case Diagram:



This is a Use Case Diagram for our PII Redaction System. It has four operations within its system: Webpage, User Interface, File Input & Processing, and File Output with Redaction changes.

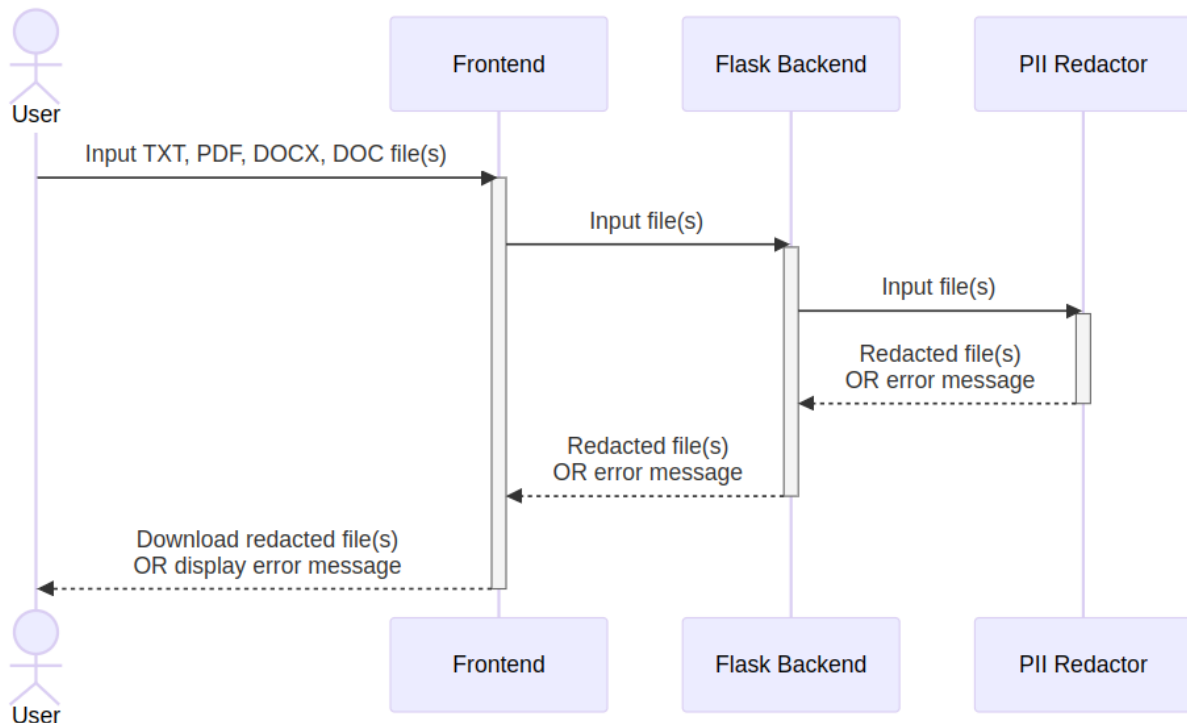
- **Webpage:** The initial visual viewed by a user. The webpage introduces the product system, helps guide the user in the proper direction, and provides the user with information regarding both the system and its developers.

- **User Interface:** The interactive interface for the user. Allows the user to navigate the webpage, make use of the system, and efficiently navigate the user through the process of using the system.
- **File Input & Processing:** Takes files from outside the system, checks for proper file formatting and then processes the files for use within the system such as converting into compatible formats for operations.
- **File Output with Redaction changes:** Receives the redacted copy, ensures the proper file formatting, and then sends it outside the system to the original inputter.

It has two actors: User and Organization.

- **User:** Individuals who need redaction for personal privacy, law requirements, or personal interests. They will be introduced to the system via the webpage and user interface. They then will interact with the input and output operations of the system, choosing which files to input, what redaction processes are desired and output formatting such as name conventions.
- **Organization:** Groups or Organizations who use redaction for mass processing often have laws and regulations, and/or policies that must be followed for their documentation. They are introduced to the system via the webpage and user interface. They then interact with the system via input and output, choosing a main redaction style for all documents to be processed. Output formatting choices for matching batch processing will also abide by chosen legal requirements and any additional format options selected by the Organizations.

Sequence Diagram:



Actors:

- User - Medical, cybersecurity, school, or government staff and other individuals that interact with our PII Redactor website.
- Frontend - The frontend component of our project that prompts the user to input file(s) they wish to redact and display the status of the redaction progress and completion status.
- Flask Backend – The backend component of our project that handles both URL routing for the frontend and executing a Python script, the PII Redactor, on the input file(s).
- PII Redactor - The Python script that redacts the given input file(s).

The user will input a document file, or files, they wish to redact into the frontend of our project. The frontend then calls the backend to pass the document files to it. The backend will execute a Python script that redacts given files which the backend will pass to, and once the Python script finishes its task or encounters an error, it will return the redacted files or error status back to the backend. The backend then returns them to the frontend in which the frontend will either display the option to download the redacted files or display an error message associated with the returned error status.

Low-Fidelity Prototype:

PII Redactor

A simple tool that safely and instantly removes all personally identifiable information (PII) from your uploaded files.

The Clever Coders

Smart Code. Clever Solutions.



Upload a file(s) (.txt, .pdf, .doc, .docx) to have the PII redacted.

No file chosen

Your file is processed securely and never stored.

Implementation Plan:

Testbed:

We will test the PII Redactor using our local computers (Windows, macOS, and Linux) to ensure proper functionality before final deployment. Testing will be done using mock documents that include various forms of PII (names, emails, phone numbers, and SSNs) as well as invalid file types to test complete coverage. The expected result is that our system correctly identifies and redacts PII, then replaces it with placeholder text without altering non-PII content while also being able to handle and correct improper use of the program.

Programming Languages:

Python (3.x) was the main programming language used to develop the back end for this application. Python was chosen for its readability, portability, and extensive library support for text processing and pattern matching. Additionally, all members of the group have experience with this language.

HTML was used for the front end of this application. HTML was chosen for its simplicity and effectiveness in creating a user interface for our application. Combined with CSS for styling, HTML gives us the ability to design the layout and user experience how we choose without complexity.

Development Tool/Device:

- **IDE:** Visual Studio or PyCharm for coding and debugging.
- **Version Control:** Git and GitHub for source-code management and collaboration.
- **Libraries:**
 - re – For regular expressions used to detect PII patterns.
 - os – For handling file operations.
 - logging – For error and activity logging.
 - SpaCy – For natural language processing.
- **Framework:** Flask for connecting HTML frontend to Python backend.